Lab 3.3: Adding a Modeless Dialog Box

In this lab, you will implement a modeless dialog box.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 3.3 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch03\Lab3.3.

The solution code for this lab is located in the folder \Labs\Ch03\Lab3.3.

Objectives

After completing this lab, you will be able to:

- Add a dialog box to an existing application.
- Implement the dialog box as a modeless dialog box.
- Respond to user actions from the dialog box in the application.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating the Dialog Box Template

In this exercise, you will create a dialog box with which the user can step through the differences between two files.

Exercise 2: Implementing the Dialog Class

In this exercise, you will implement the dialog class, **CFindDifferenceDialog**, which you created in Exercise 1. Because this dialog box is modeless, all its actions will take place while its window still exists. Therefore, you will not need to use DDX synchronization; you can query controls directly.

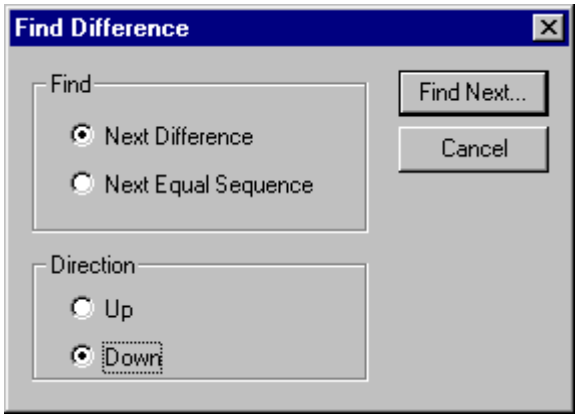
Exercise 3: Integrating the Dialog Box into an Application

In this exercise, you will provide the code that responds to user actions in the dialog box.

77 Exercise 1: Creating the Dialog Box Template

The code that forms the starting point for this exercise is in \Labs\Ch03\Lab3.3\Ex01.

In this exercise, you will create a dialog box with which the user can step through the differences between two files. The following illustration shows the dialog box you will create.



There are three parts to this exercise:

☐ 1. Creating the dialog box template.
☐ 2. Adding the controls to the dialog box template.
☐ 3. Testing the dialog box and setting the tab order.

▶ Create a new (unadorned) dialog box resource

☐ 1. On the **Resource** toolbar, click the **New Dialog** button to create a dialog box template.
☐ 2. Set the Caption and ID properties of the dialog box as follows.

Property	Value
Caption	Find Difference
ID	IDD_NEXTDIFF

(CD-ROM displays the following tip.)

For the static text controls, the ampersand (&) in the static text establishes ALT key access to the control that follows in the tab order. A static control cannot have focus, so focus automatically flows to the next control in the tab-order sequence. In this case, the shortcut keys in the static-text labels provide access to the associated edit-box controls.

Notes

▶ Add controls to the dialog box template

☐ 1. Open the IDD_NEXTDIFF dialog box resource.

☐ 2. Add the following controls to the dialog box to create a dialog box as shown in the preceding illustration.

Control type	ID	Caption
Default command button (push button)	IDOK	Find Next...
Command button	IDCANCEL	Cancel
Group box	IDC_STATIC	Find
Option button	IDC_RADIO_NEXTDIFF	&Next Difference
Option button	IDC_RADIO_NEXTEQUAL	Next &Equal Sequence
Group box	IDC_STATIC	Direction
Option button	IDC_RADIO_UP	&Up
Option button	IDC_RADIO_DOWN	&Down

▶ Group controls on the dialog box template

☐ 1. Right-click the **Next Differences** option button and click **Properties**.

☐ 2. Click **Group**.

☐ 3. Double-click the **Up** option button and click its **Group** box.

☐ 4. Double-click the **Find Next** button and click its **Group** box.

☐ 5. Save the dialog box template.

▶ Set the tab order for the controls on the dialog box template

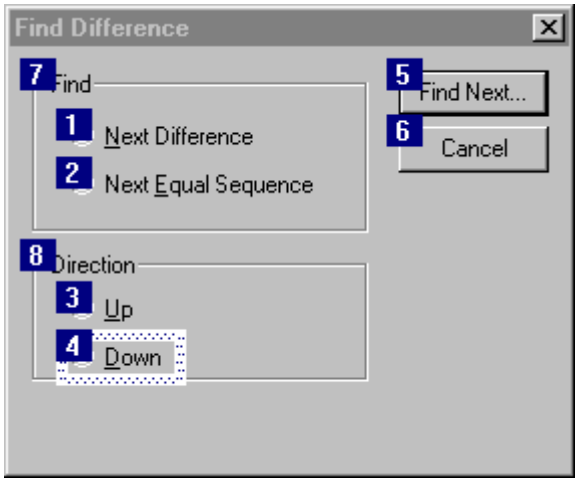
☐ 1. On the **Layout** menu, click **Tab Order**.

The property page is hidden. The dialog editor now displays a number for each of the controls in your dialog box template. By default, the numbers indicate the order in which each control was added to the template. While setting the tab order, note that the **OK** button automatically has the **Default Button** option selected.

Notes

☐ 2. To set the tab order, click the controls in the appropriate order. For example, first click the **Next Difference** option button to set it as the first control, then click **Next Equal Sequence**, and so forth.

Set the tab order as shown in the following illustration.



☐ 3. To end the tab-ordering operation, click inside the dialog editor window, but outside the dialog box resource. (Pressing ESC also will end the session.)

▶ **Test the dialog box resource**

In this procedure, you will test your controls.

☐ 1. Click the **Test** button on the **Dialog** toolbar.

A simulation of the dialog box created from this resource appears.

Note the control that has initial keyboard focus. Also note that the **Find Next** button is the default button. It is activated when the user presses the ENTER key.

☐ 2. Test the shortcut-key sequences.

☐ 3. Press the TAB key several times to cycle through the controls and verify that the tab order is correct.

☐ 4. Within the option button group, use the arrow keys to navigate among the buttons.

☐ 5. Click the **Find Next** or **Cancel** button to exit test mode.

▶ **Check and save your file**

☐ 1. Test the dialog box template again.

☐ 2. Save the current file when you are satisfied with your work.

Notes

▶ Create a dialog class using ClassWizard

In this procedure, you will use ClassWizard to create a dialog class in the Diff application.

(CD-ROM displays the following tip.)

To have ClassWizard automatically display the **Adding A Class** dialog box, open ClassWizard during the dialog editor session in which you create the new dialog resource.

☐ 1. Run the dialog editor on the IDD_NEXTDIFF dialog box resource.

☐ 2. In the dialog editor, be sure that your dialog box template window is the active child window.

☐ 3. Invoke ClassWizard.

It should automatically display the **Adding A Class** dialog box. (If this dialog box does not appear, click the **Add Class** button).

☐ 4. Click **Create a new class**, and click **OK**.

The **Create New Class** dialog box appears.

☐ 5. In the **Name** box, type **CFindDifferenceDialog** for the name of the associated C++ dialog class.

☐ 6. In the **Base Class** box, click **CDialog**.

The Dialog ID should already be set to IDD_NEXTDIFF.

☐ 7. Shorten the file names FindDifferenceDialog.cpp and FindDifferenceDialog.h to FindDiff.cpp and FindDiff.h by using the **Change** button and its dialog box, and then click **OK**.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.3\Ex01\Solution.

Notes

77 Exercise 2: Implementing the Dialog Class

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch03\Lab3.3\Ex02.

In this exercise, you will implement the dialog class, **CFindDifferenceDialog**, which you created in the previous exercise. Because this dialog box is modeless, all its actions will take place while its window still exists. Therefore, you will not need to use DDX synchronization; you can query controls directly.

A modeless dialog box has its own stand-alone window with its own window procedure. A modeless dialog box can communicate with its parent window in many ways; the simplest (and safest) is to use registered window messages known by the dialog class and the main application.

▶ **Set up members for control states in the dialog box**

☐ 1. Open the file FindDiff.h.

☐ 2. Create member variables to determine whether terminating or finding next difference:

```
//attributes
protected:
    BOOL m_bTerminating;
    BOOL m_bFindNext;
```

☐ 3. Declare member functions to return the internal state to the application as follows:

```
public:
    BOOL        IsTerminating() const;  //TRUE if terminating
    BOOL        SearchDown() const;     //TRUE if searching down,
                                        //FALSE if searching up
    BOOL        FindDifference() const; //TRUE if finding next difference
                                        //FALSE if finding next equal
    BOOL        FindNext() const;       //TRUE if find next button pressed
```

☐ 4. Save FindDiff.h.

☐ 5. Open the file FindDiff.cpp.

☐ 6. Implement the **CFindDifferenceDialog::IsTerminating** function to return m_bTerminating as follows:

```
BOOL CFindDifferenceDialog::IsTerminating() const
{
    return m_bTerminating;
}
```

☐ 7. Implement **CFindDifferenceDialog::SearchDown** to return the state of the **Search Down** option button as follows:

```
BOOL CFindDifferenceDialog::SearchDown() const
{
    return(IsDlgButtonChecked(IDC_RADIO_DOWN));
}
```

Notes

☐ 8. Implement **CFindDifferenceDialog::FindDifference** to return the state of the **Next Difference** option button as follows:

```
BOOL CFindDifferenceDialog::FindDifference() const
{
    return(IsDlgButtonChecked(IDC_RADIO_NEXTDIFF));
}
```

☐ 9. Implement **CFindDifferenceDialog::FindNext** to return m_bFindNext as follows:

```
BOOL CFindDifferenceDialog::FindNext() const
{
    return m_bFindNext;
}
```

☐ 10. Save FindDiff.cpp.

▶ Implement the Find Next button handler

When the main application window is notified that the user has clicked a button, it will query the dialog box about the action. You could use the WPARAM parameter of the message to pass this information, but this would entail a more complicated maintenance scheme.

☐ 1. Open FindDiff.h.

☐ 2. Define a constant string for the name of the message as follows:

```
const char* const FINDDIFF_MSGSTRING = "diffapp_FindDifference";
```

☐ 3. Save FindDiff.h.

☐ 4. Invoke ClassWizard.

☐ 5. Create a handler for BN_CLICKED on the IDOK button. Give it the name **OnFindNext**.

☐ 6. In the **OnFindNext** function, set m_bFindNext to **TRUE** as follows:

```
m_bFindNext = TRUE;
```

☐ 7. Send the private message to the parent of the dialog box with a pointer to the dialog object in LPARAM:

```
GetParent()->SendMessage(
    ::RegisterWindowMessage(FINDDIFF_MSGSTRING),
    0, (LPARAM)this);
```

☐ 8. Reset m_bFindNext as follows:

```
m_bFindNext = FALSE;
```

For more information about creating a communication link between the dialog box and its parent application, see Communicating with the Application in this chapter.

Notes

▶ Implement dialog initialization

☐ 1. In the constructor, initialize m_bTerminating and m_bFindNext to **FALSE** as follows:

```
CFindDifferenceDialog::CFindDifferenceDialog(CWnd* pParent /*=NULL*/)
    : CDialog(CFindDifferenceDialog::IDD, pParent)
{
    //{{AFX_DATA_INIT(CFindDifferenceDialog)
        // NOTE: ClassWizard will add member initialization here
    //}}AFX_DATA_INIT

    m_bTerminating = FALSE;
    m_bFindNext = FALSE;
}
```

☐ 2. Invoke ClassWizard. Create handlers for WM_INITDIALOG message and **Create**.

☐ 3. In the **OnInitDialog** handler, click the **Next Difference** and **Down** option buttons as follows:

```
BOOL CFindDifferenceDialog::OnInitDialog()
{
    CDialog::OnInitDialog();

    //  Our initialization
    CheckDlgButton(IDC_RADIO_NEXTDIFF, TRUE);
    CheckDlgButton(IDC_RADIO_DOWN, TRUE);

    return TRUE;  // return TRUE unless you set the focus to a control
                  // EXCEPTION: OCX Property Pages should return FALSE
}
```

☐ 4. Delete the parameters from the **Create** handler. Because all the members are set up in **CDialog::CDialog**, the call to the **Create** function uses these variables:

```
BOOL CFindDifferenceDialog::Create()
{
    //  m_lpszTemplateName and m_pParent are set
    //  up by CDialog during construction
    return CDialog::Create (m_lpszTemplateName, m_pParentWnd);
}
```

☐ 5. Save FindDiff.cpp.

☐ 6. In the file FindDiff.h, remove the parameters from **Create**.

☐ 7. Save FindDiff.h.

For more information about creating a modeless dialog box, see Initializing and Displaying a Modeless Dialog Box in this chapter.

Notes

▶ Implement dialog shutdown

☐ 1. Use ClassWizard to create handlers for BN_CLICKED on the IDCANCEL button (**OnCancel**) and the **PostNcDestroy** messages.

☐ 2. Edit the code of **CFindDifferenceDialog::OnCancel**. Use **CWnd::DestroyWindow** to close the window when the **Cancel** button is pressed:

```
void CFindDifferenceDialog::OnCancel()
{
    DestroyWindow();
}
```

☐ 3. Edit the code of **CFindDifferenceDialog::PostNcDestroy**. **PostNcDestroy** is sent by **CWnd** after the window has been destroyed. Because your window no longer exists, you cannot follow your window tree with **CWnd::GetParent**. You will use the stored m_mParentWnd handle to communicate with the parent window. Set the m_bTerminating member to **TRUE**, and then send the message to your parent window:

```
m_bTerminating = TRUE;
m_pParentWnd->SendMessage(
    ::RegisterWindowMessage(FINDDIFF_MSGSTRING),
    0, (LPARAM)this);
```

☐ 4. As the last step, delete your object:

```
delete this;
```

☐ 5. Save FindDiff.cpp.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CFindDifferenceDialog::PostNcDestroy()
{
    m_bTerminating = TRUE;
    m_pParentWnd->SendMessage(
        ::RegisterWindowMessage(FINDDIFF_MSGSTRING),
        0, (LPARAM)this);

    delete this;
}
```

☐ 6. Build your project.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.3\Ex02\Solution.

Notes

77 Exercise 3: Integrating the Dialog Box into an Application

Continue with the files you created in Exercise 2 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch03\Lab3.3\Ex03.

A modeless dialog box shares code with its parent window. In this exercise, you will provide the code that responds to actions in the dialog box. This code is placed in the **CMainFrame** class for simplicity; however, any **CWnd**-derived window (such as **CDiffView**) could be used as the target.

▶ Resolve dependencies on CDifferenceDialog in Mainfrm.h

- You will add reference to **CFindDifferenceDialog** class in MainFrm.h. You will need to include FindDiff.h in any file that includes MainFrm.h prior to that include. Add the include statements to these files.
  - MainFrm.cpp
  - DiffView.cpp
  - DiffDoc.cpp
  - Diff.cpp

▶ Add menu items to the IDR_MAINFRAME menu

☐ 1. Open the IDR_MAINFRAME menu.

☐ 2. Add a separator to the end of the **Edit** menu.

☐ 3. Add these two menu items after the separator:

ID	Caption
ID_EDIT_FIND	&Find...
ID_EDIT_FINDDIFF	Find &Difference...

☐ 4. Save Diff.rc.

▶ Integrate CFindDifferenceDialog into CMainFrame

☐ 1. Open MainFrm.h.

☐ 2. Declare a pointer to a **CFindDifferenceDialog** object in the protected implementation section:

```
CFindDifferenceDialog*  m_pFindDiffDlg;
```

☐ 3. Declare a handler for the FINDDIFF_MSGSTRING registered message before DECLARE_MESSAGE_MAP as follows:

```
afx_msg LRESULT OnFindDifferenceCmd (WPARAM, LPARAM lParam);
```

☐ 4. Declare a member function to find the next difference as follows:

```
void OnFindNextDifference(BOOL bSearchDown,
                          BOOL bNextDifference);
```

☐ 5. Save MainFrm.h.

☐ 6. Open MainFrm.cpp.

Notes

☐ 7. Initialize a variable to hold the ID of the registered message:

```
static const UINT nMsgFindDifference =
                    ::RegisterWindowMessage(FINDDIFF_MSGSTRING);
```

☐ 8. Map the message to **CMainFrame** as follows:

```
ON_REGISTERED_MESSAGE (nMsgFindDifference, OnFindDifferenceCmd)
```

☐ 9. Using ClassWizard, add a command handler for ID_EDIT_FINDDIFF (**OnEditFindDiff**).

☐ 10. In the constructor, set m_pFindDiffDlg to NULL as follows:

```
m_pFindDiffDlg = NULL;
```

▶ **Implement the menu handler for ID_EDIT_FINDDIFF**

☐ 1. Edit the code for **CMainFrame::OnEditFindDiff**. Check to see whether the dialog box is already displayed:

```
if(m_pFindDiffDlg == NULL)
```

☐ 2. Dynamically construct a **CFindDifferenceDialog** instance and assign the pointer to m_pFindDiffDlg if the dialog box is not already displayed:

```
m_pFindDiffDlg = new CFindDifferenceDialog(this);
```

☐ 3. Once you have constructed the dialog box, call **CFindDifferenceDialog::Create** to initialize it as follows:

```
if(m_pFindDiffDlg)
{
    m_pFindDiffDlg->Create();
}
```

☐ 4. Display the dialog window as follows:

```
if(m_pFindDiffDlg)
{
    m_pFindDiffDlg->SetActiveWindow();
    m_pFindDiffDlg->ShowWindow(SW_SHOW);
}
```

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CMainFrame::OnEditFindDiff()
{
    //  Create the dialog if needed

    if(m_pFindDiffDlg == NULL)
    {

        m_pFindDiffDlg = new CFindDifferenceDialog(this);
        if(m_pFindDiffDlg)
        {
            m_pFindDiffDlg->Create();
        }
    }

    //  Show it

    if(m_pFindDiffDlg)
    {
        m_pFindDiffDlg->SetActiveWindow();
        m_pFindDiffDlg->ShowWindow(SW_SHOW);
    }
}
```

▶ **Implement a handler for the registered message**

☐ 1. In the MainFrm.cpp file, define **CMainFrame::OnFindDifferenceCmd**:

```
LRESULT CMainFrame::OnFindDifferenceCmd(WPARAM, LPARAM lParam)
```

☐ 2. Cast LPARAM to a pointer to **CFindDifferenceDialog**:

```
CFindDifferenceDialog* pDialog = (CFindDifferenceDialog *)lParam;
```

☐ 3. The dialog box sends its message when the **Find Next** button is clicked, or when the dialog box is closing. In the latter case, clear m_pFindDiffDlg:

```
if (pDialog->IsTerminating())
{
    m_pFindDiffDlg = NULL;
}
```

☐ 4. When the **Find Next** button is clicked, check the state of the option buttons and dispatch to **CMainFrame::OnFindNextDifference** as follows:

```
else if (pDialog->FindNext())
{
    OnFindNextDifference(pDialog->SearchDown(),
                         pDialog->FindDifference());
}
```

☐ 5. Return 0 for the message.

Notes

☐ 6. Save MainFrm.cpp.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
LRESULT CMainFrame::OnFindDifferenceCmd(WPARAM, LPARAM lParam)
{

    CFindDifferenceDialog* pDialog = (CFindDifferenceDialog *)lParam;

    if (pDialog->IsTerminating())
    {
        m_pFindDiffDlg = NULL;
    }
    else if (pDialog->FindNext())
    {
        OnFindNextDifference(pDialog->SearchDown(),
                             pDialog->FindDifference());
    }

    return 0;
}
```

▶ **Implement visual feedback**

Since the difference is simulated, finding the next difference also will need to be simulated. You will simply move randomly forward or backward through the current view in response to the message from the dialog box.

☐ 1. In the MainFrm.cpp file, define **CMainFrame::OnFindNextDifference**:

```
void CMainFrame::OnFindNextDifference(BOOL bSearchDown,
                                      BOOL bNextDifference)
```

☐ 2. Get the active view:

```
CDiffView * pView = (CDiffView *)GetActiveView();
if(pView)
{
```

☐ 3. Find out which line (if any) is currently highlighted:

```
int nLineCnt = pView->GetRichEditCtrl().GetLineCount();
LONG lStart = 0;
LONG lEnd   = 0;
pView->GetRichEditCtrl().GetSel(lStart, lEnd);
LONG lCurLine = pView->GetRichEditCtrl().LineFromChar(lStart);
```

Notes

☐ 4. Find a line randomly between the current line and the end of the text if you are searching forward; find a line randomly between the current line and the beginning of the text if you are searching backward:

```
int nNewLine;
if(bSearchDown)
{
    nNewLine = lCurLine + (rand() % (nLineCnt-lCurLine)+1);
}
else
{
    nNewLine = rand() % (lCurLine+1) + 1;
}
```

☐ 5. Find the starting and ending characters of that line and select them as follows:

```
lStart = pView->GetRichEditCtrl().LineIndex(nNewLine);
lEnd = lStart + pView->GetRichEditCtrl().LineLength(nNewLine);

pView->GetRichEditCtrl().SetSel(lStart, lEnd);
```

☐ 6. There is no difference between the next difference and the next equal text in this simulation. Use the status bar text to show the current option as follows:

```
if (bNextDifference)
{
    m_wndStatusBar.SetWindowText(_T("Found next difference"));
}
else
{
    m_wndStatusBar.SetWindowText(_T("Found next equal run"));
}
```

Notes

☐ 7. Save MainFrm.cpp.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CMainFrame::OnFindNextDifference(BOOL bSearchDown,
                                      BOOL bNextDifference)
{
    CDiffView * pView = (CDiffView *)GetActiveView();
    if(pView)
    {
        int nLineCnt = pView->GetRichEditCtrl().GetLineCount();
        LONG lStart = 0;
        LONG lEnd   = 0;
        pView->GetRichEditCtrl().GetSel(lStart, lEnd);
        LONG lCurLine = pView->GetRichEditCtrl().LineFromChar(lStart);

        int nNewLine;
        if(bSearchDown)
        {
            nNewLine = lCurLine + (rand() % (nLineCnt-lCurLine)+1);
        }
        else
        {
            nNewLine = rand() % (lCurLine+1) + 1;
        }

        lStart = pView->GetRichEditCtrl().LineIndex(nNewLine);
        lEnd = lStart + pView->GetRichEditCtrl().LineLength(nNewLine);

        pView->GetRichEditCtrl().SetSel(lStart, lEnd);

        if (bNextDifference)
        {
            m_wndStatusBar.SetWindowText(_T("Found next difference"));
        }
        else
        {
            m_wndStatusBar.SetWindowText(_T("Found next equal run"));
        }
    }
}
```

▶ **Build and test the application**

- Build your application. Choose menu items under **Edit**, set the focus in either pane, and find the next difference between the files either forward or backward.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.3\Ex03\Solution.

Notes